



COS6012-B: Concurrent and Distributed Systems

Introduction to Concurrency

Dr Daniele Scrimieri

Outline

- Introduction to concurrency
- Concurrent and distributed software
- Types of concurrent systems
- Applications of concurrency

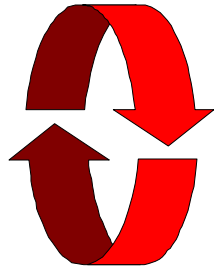
What is Concurrency?

- Computers appear to do many things at once
 - running multiple programs on a laptop
 - writing back data buffered in memory to the hard disk while the program(s) continue to execute
- Concurrency means **multiple computations** are happening **at the same time**
- Concurrency is essential in modern programming:
 - web sites must handle multiple simultaneous users
 - mobile apps need to do some of their processing on servers (“in the cloud”)
 - graphical user interfaces almost always require background work that does not interrupt the user

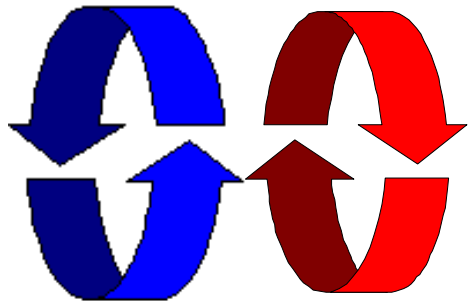
Examples of Concurrency

- **GUI-based applications**
- **Mobile apps**
- **Web services:** HTTP daemons, servlet engines, application servers
- **Component-based software:** Java beans often use threads internally
- **I/O processing:** concurrent programs can use time which would otherwise be wasted waiting for slow I/O
- **Real time systems:** operating systems, transaction processing systems, industrial process control, embedded systems, etc.
- **Parallel processing:** simulation of physical and biological systems, graphics, economic forecasting, etc.

Sequential vs Concurrent Program



A **sequential** program has a *single thread* of control.



A **concurrent** program has *multiple threads* of control allowing it perform multiple computations in parallel and to control multiple external activities which occur at the same time.

A Non-Sequential Example

- Consider a house with a burglar alarm system.
- The system continually monitors:
 - ✓ The front door
 - ✓ The back door
 - ✓ The sliding glass door
 - ✓ The door to the deck
 - ✓ The kitchen windows
 - ✓ The living room windows
 - ✓ The bedroom windows
- The burglar alarm is watching all of these “at once” (**at the same time**).

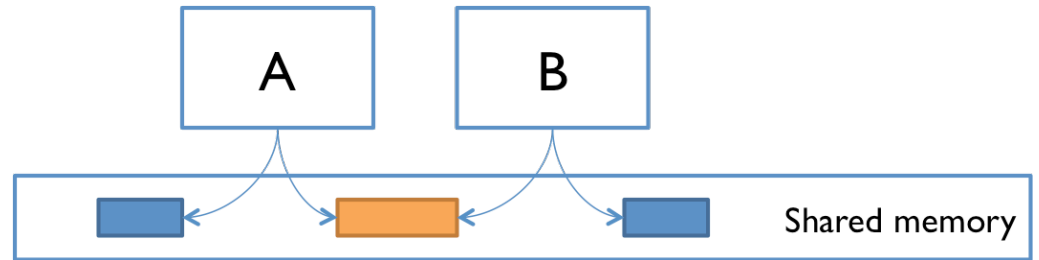
Concurrent Systems

A system in which:

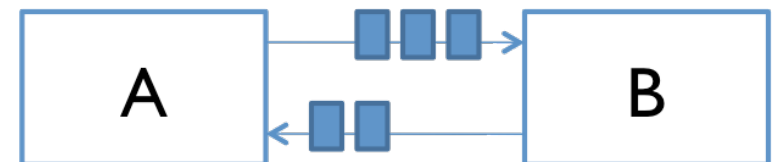
- ✓ Multiple tasks can be **executed at the same time**.
- ✓ The tasks may be **duplicates** of each other, **or distinct** tasks.
- ✓ The overall time to perform the series of tasks is **reduced**.

Implementations of Concurrency

- **Shared memory:** the execution of concurrent processes by running them on one or more processors all of which access a shared memory - processes communicate by reading and writing shared memory locations;



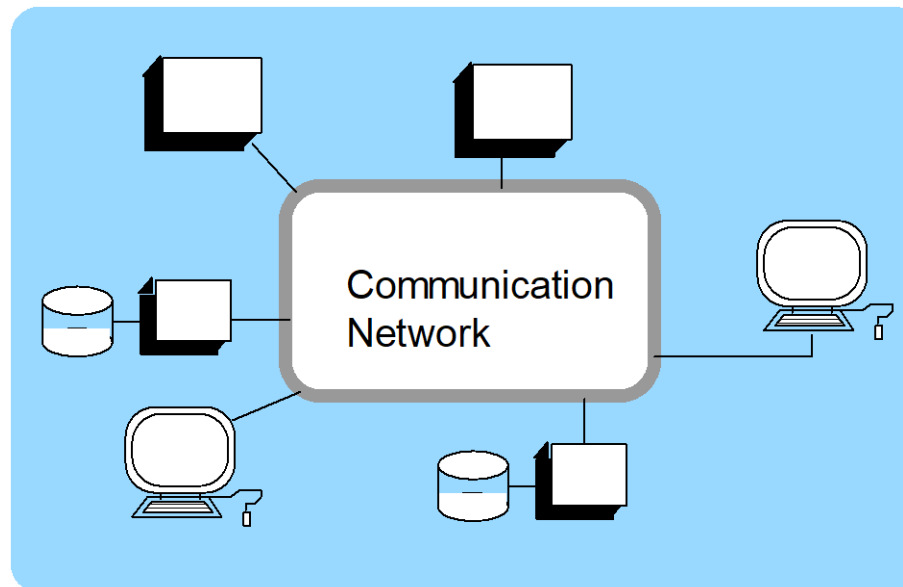
- **Message passing (distributed processing):** the execution of concurrent processes by running them on separate processors - processes communicate by message passing



Concurrent and Distributed Software

Interacting, concurrent software components of a system:

- single machine -> **shared memory interactions**
- multiple machines -> **network interactions**



Advantages of Concurrency

- ✓ Concurrent processes can **reduce duplication** in code
- ✓ The overall **runtime** of the algorithm can be significantly reduced
- ✓ More **real-world problems** can be solved than with sequential algorithms alone
- ✓ Concurrent programs are often more flexible or **responsive** than single-threaded programs
- ✓ A concurrent design may make better use of system resources



Disadvantages of Concurrency

- ☹️ **Runtime is not always reduced**, so careful planning is required.
- ☹️ Concurrent algorithms can be **more complex** than sequential algorithms.
- ☹️ Shared data can be **corrupted**.
- ☹️ **Communication** between tasks needed.
- ☹️ Concurrency introduces new kinds of **bugs**.



Concurrency vs. Parallelism

- **Concurrency** is the execution of multiple tasks at the same time, **regardless of the number of processors.**
- **Parallelism** is the execution of the same task (**data parallelism**) or multiple tasks (**task parallelism**) on **multiple processors.**
- Data parallelism involves:
 - Splitting the data into meaningful pieces.
 - Doing the work on many processors.
 - Coordinating and putting the pieces back together.

Concurrency vs. Parallelism

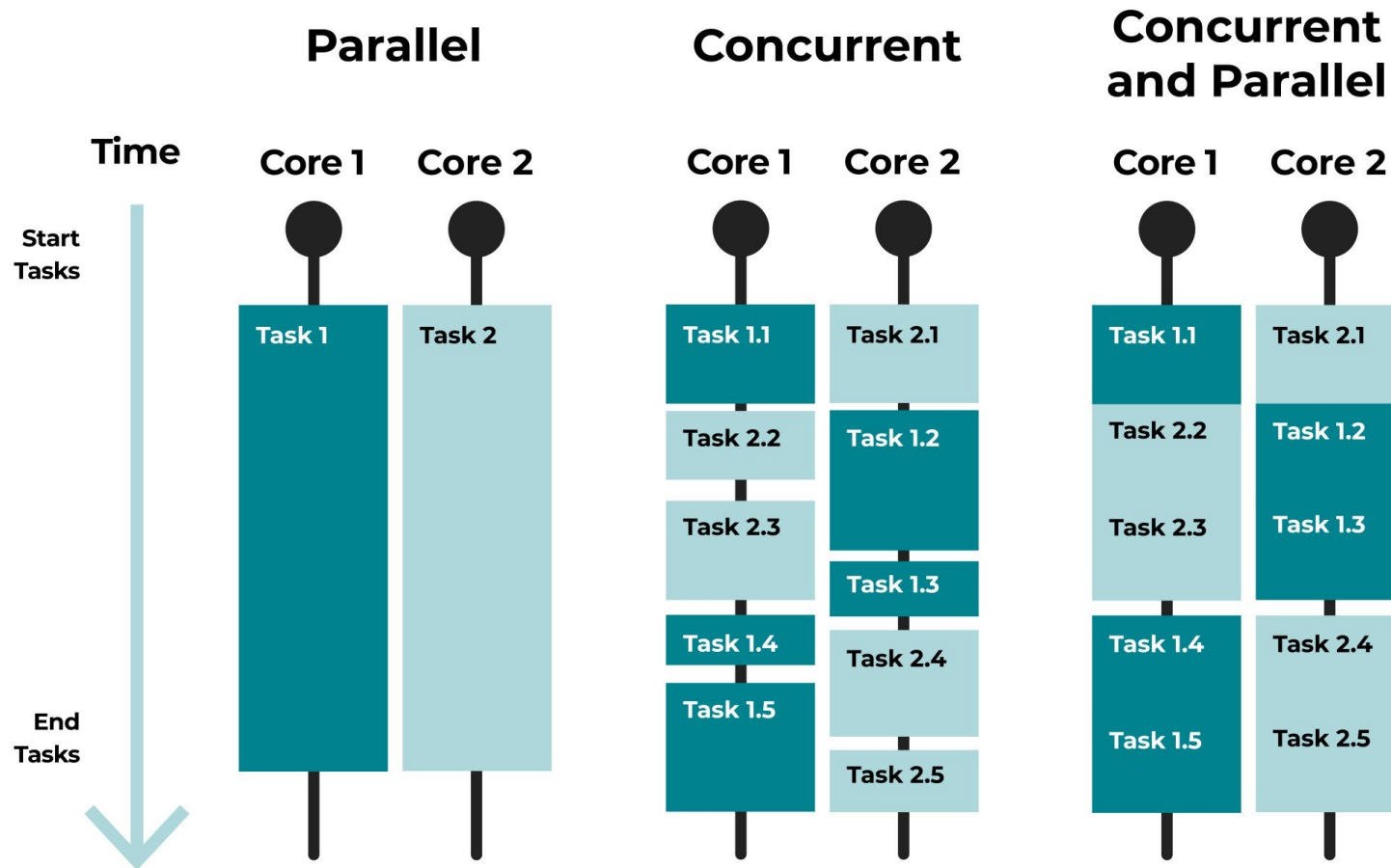


Image: <https://openclassrooms.com/en/courses/5684021-scale-up-your-code-with-java-concurrency/5684028-identify-the-advantages-of-concurrency-and-parallelism>

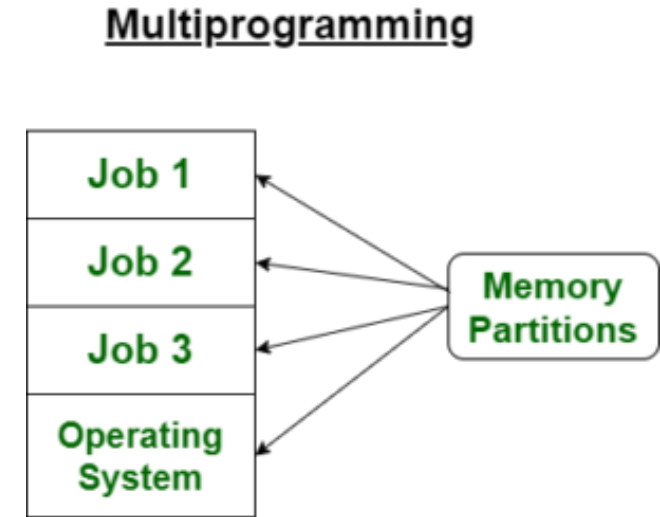
Types of Concurrent Systems

- Multiprogramming
- Multitasking
- Multiprocessing
- Distributed Systems

Multiprogramming and Multitasking

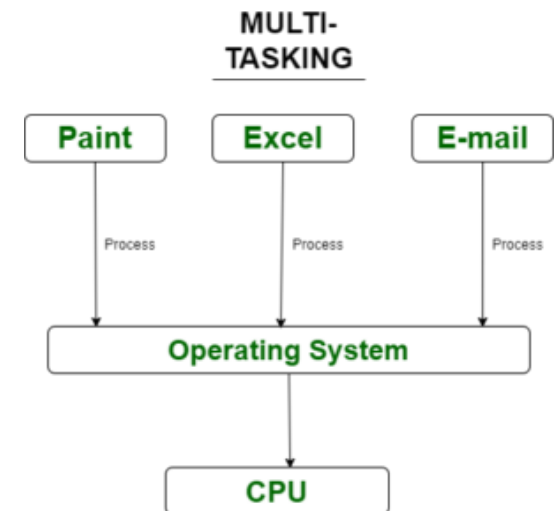
Multiprogramming

- Shares a **single processor** among many users or tasks.
- May have a time-sharing algorithm or a priority algorithm for determining which task to run next.



Multitasking

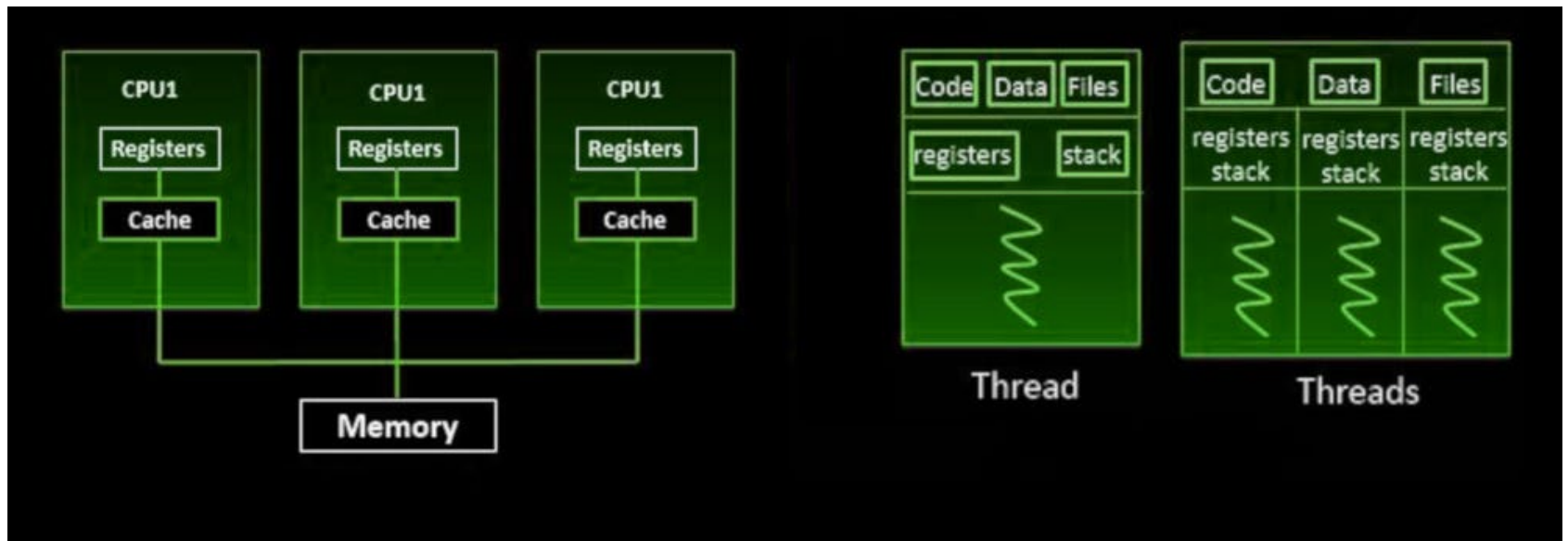
- Gives the illusion of simultaneous processing through **rapid swapping of tasks (interleaving)**.



Images: <https://www.geeksforgeeks.org/difference-between-multiprogramming-and-multitasking/>

Multiprocessing

- Executes **multiple tasks** at the same time.
- Uses **multiple processors** to accomplish the tasks.
- Each processor may also **timeshare** among several tasks.
- Has a **shared memory** that is used by all the tasks.



Distributed Systems

- Multiple computers working **together** with no central program “in charge.”

- Advantages:**

- ☺ No bottlenecks from sharing processors
- ☺ No central point of failure
- ☺ Processing can be localised for efficiency



- Disadvantages:**

- ☹ Complexity
- ☹ Communication overhead
- ☹ Distributed control



Why Concurrent Programming?

- **Performance** gain from multiprocessing hardware
- Increased application throughput: **avoid polling (busy waiting)!**
 - an I/O call needs only to block one thread
- Increased application **responsiveness**
 - high priority thread for user requests
- **More appropriate structure**
 - for programs which interact with the environment, control multiple activities and handle multiple

Concurrency is widespread but error prone!

A very simple example:

- ◆ process 1: $x := x + 1$ (x shared variable)
- ◆ process 2: $x := x - 1$ (x shared variable)

Final result?

- Single line instructions are generally **not atomic**.
- Assuming read and write are atomic, the result depends on the order of read and write operations on x!

We have a race condition!

Concurrency is widespread but **error** prone!

- There are **4 atomic** x-operations:
Process 1 reads x (R1), writes to x (W1) $x := x + 1$
Process 2 reads x (R2), writes to x (W2) $x := x - 1$
- R1 must happen before W1 and R2 before W2, so these operations can be sequenced in 6 ways (x initially 0):

R1	R1	R1	R2	R2	R2
W1	R2	R2	R1	R1	W2
R2	W1	W2	W1	W2	R1
W2	W2	W1	W2	W1	W1
0	-1	1	-1	1	0

- We see that the final value of x is **-1, 0, or 1**. The program is thus **non-deterministic**: the result can vary from execution to execution.

Concurrency is widespread but **error** prone!

- **Therac-25** computerised radiation therapy machine:
 - It was involved in at least six accidents between 1985 and 1987, in which patients were given massive overdoses of radiation.
 - Because of concurrent programming errors (also known as race conditions), it sometimes gave its patients radiation doses that were hundreds of times greater than normal, resulting in **death or serious injury**
<https://en.wikipedia.org/wiki/Therac-25>
- **Mars Rover**
 - Problems with interaction between concurrent tasks caused periodic software resets reducing availability for exploration.



What to do?

- Designing concurrent software can be complex and **error** prone.
- How to prove the **correctness** of concurrent algorithms?
- A **rigorous approach** is essential.
- **Verification** (model checking): deadlock, safety, progress, liveness, starvation.
 - Models are needed to experiment, test, and check a design before it is implemented – specification languages.
 - Theories for Concurrent Programming - Process Algebras:
 - CCS (Calculus of Communicating Systems)
 - CSP (Communicating Sequential Processes)
 - FSP (Finite State Processes)
- **Testing** - new challenges!
- **Design patterns** for common concurrency problems.

Turing Awards related to verification (model checking, state machines, etc.)

(2007) Edmund M. Clarke, E. Allen Emerson, Joseph Sifakis:

- For their roles in developing **model checking** into a highly effective **verification** technology, widely adopted in the hardware and software industries.

(1996) Amir Pnueli:

- For seminal work introducing **temporal logic** into computing science and for outstanding contributions to program and system **verification**.

(1986) John E Hopcroft, Robert Tarjan:

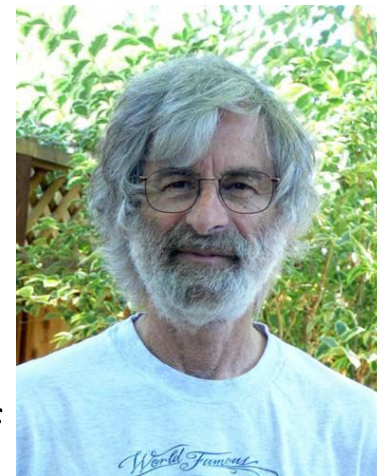
- Foundation of formal languages for **state machines**

(1976) Michael O. Rabin, Dana S. Scott:

- For their joint paper "Finite Automata and Their Decision Problem," which introduced the idea of **nondeterministic machines**, which has proved to be an enormously valuable concept.

Turing Awards related to Concurrency and Process Algebras

(2013) Leslie Lamport: Fundamental contributions to theory and practice of **distributed and concurrent systems**, notably the invention of concepts such as causality and logical clocks, safety and liveness, replicated state machines, and sequential consistency.



(1991) Robin Milner: For three distinct and complete achievements: (1) LCF the mechanization of Scott's Logic of Computable Functions, for machine assisted proof construction; (2) ML the first language to include polymorphic type inference together with a type-safe exception-handling mechanism; (3) **Process Algebra: CCS (Calculus of Communicating Systems) a general theory of concurrency.**



(1980) Tony Hoare: For his fundamental contributions to the definition and design of programming languages. **Process Algebra:** the language and theory around **CSP (Communicating Sequential Processes)**



Acknowledgments

Lecture notes / slides

- Dr. Brian Logan, University of Nottingham, UK
- Dr. Rajkumar Buyya, The University of Melbourne, Australia
- Dr. David J. Greaves and Dr. Martin Kleppmann, University of Cambridge, UK
- Prof. Irfan Awan, University of Bradford, UK

Books

- Magee & Kramer (2006). Concurrency: State Models and Java Programs (2nd edition), John Wiley.

